ALGORYTMY I STRUKTURY DANYCH

WYŻSZA SZKOŁA EKONOMICZNA W BIAŁYMSTOKU

INFORMATYKA STOSOWANA

MGR INŻ. GRZEGORZ ALEKSIUK

ANALIZA ZŁOŻONOŚCI

INTUICJA: WYSZUKIWANIE ELEMENTU W TABLICY

Wyszukiwanie liniowe

- Elementy listy nie muszą być posortowane
- Bardzo prosty
- Aby znaleźć element trzeba przejść po wszystkich elementach

Wyszukiwanie binarne

- Elementy listy muszą być posortowane
- Bardziej skomplikowany
- Aby znaleźć element trzeba wykonać znacząco mniej operacji (ale ile konkretnie?)

NOTACJA O(N)

- Jest to notacja matematyczna opisująca asymptotyczne zachowanie funkcji.
- Określa górną granicę dla złożoności czasowej lub pamięciowej algorytmu.
- Wskazuje, jak rośnie czas wykonania wraz ze wzrostem rozmiaru danych.
- Pozwala na porównywanie wydajności różnych algorytmów.

RODZAJE ZŁOŻONOŚCI

Obliczeniowa (czasowa)

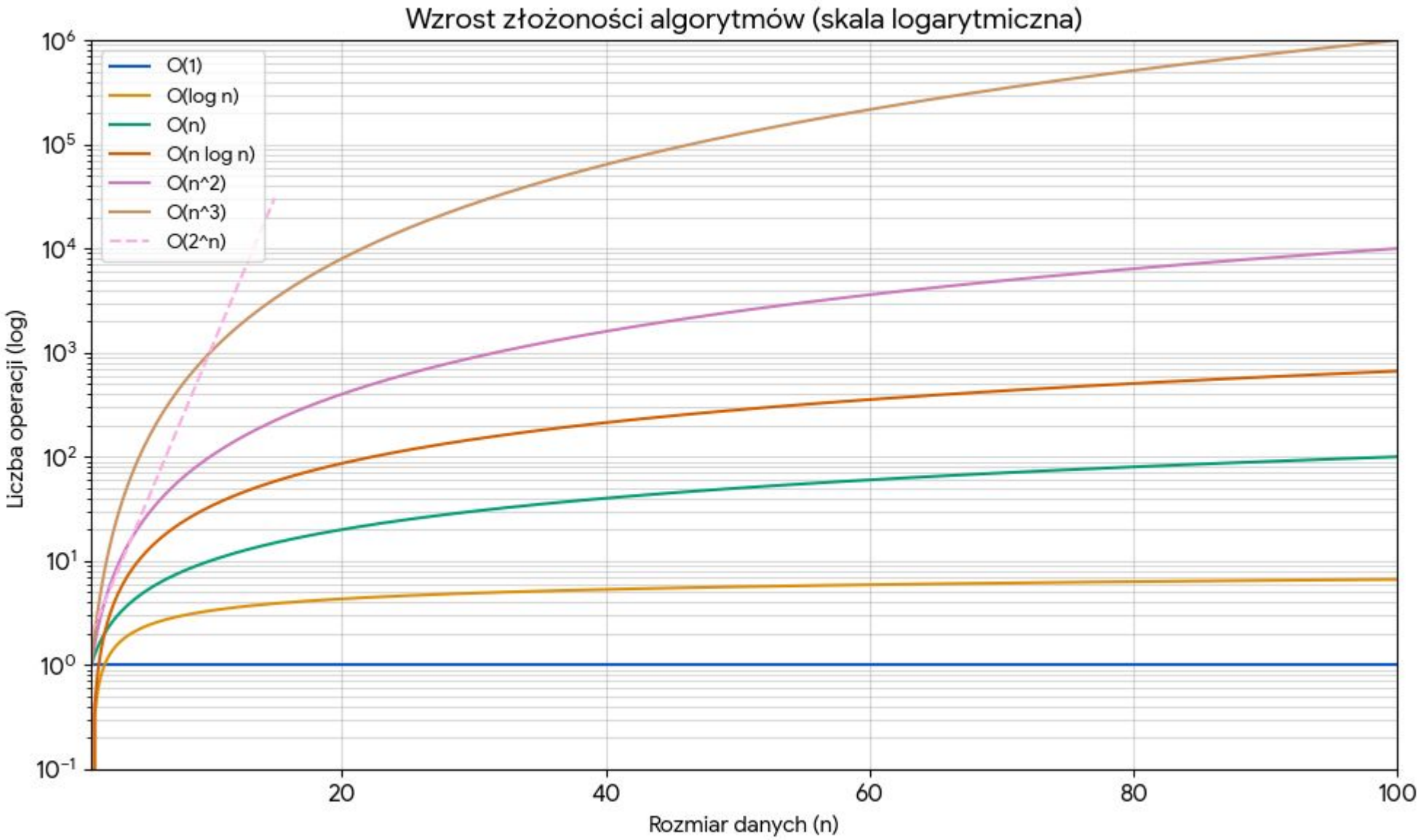
- Określa, jak czas wykonania algorytmu rośnie wraz ze wzrostem rozmiaru danych wejściowych.
- Liczba operacji dominujących: np. porównania, podstawienia.
- Przypadek pesymistyczny: Najczęściej analizowany scenariusz "najgorszej możliwości".

Pamięciowa (przestrzenna)

- Określa ilość pamięci operacyjnej potrzebnej do wykonania algorytmu w funkcji rozmiaru danych.
- Struktury danych tworzone w trakcie działania (tablice, drzewa).

PRZYKŁADY ZŁOŻONOŚCI OBLICZENIOWEJ

- O(1) - stała - sprawdzenie parzystości liczby
- O(log n) - logarytmiczna – znajdywanie elementy w posortowanej tablicy
- O(n) - liniowa – znajdywanie elementu w nieposortowanej tablicy
- O(n log n) - liniowo-logarytmiczna – sortowanie przez scalanie
- $O(n^2)$ - kwadratowa – naiwne znajdywanie duplikatów w tablicy
- $O(n^3)$ - sześcienna - mnożenie kwadratowych macierzy
- $O(2^n)$ - wykładnicza - znajdywanie wszystkich podzbiorów
- O(n!) - silniowa – znajdywanie wszystkich permutacji elementów tablicy


Wzrost złożoności algorytmów (skala logarytmiczna)
O(1)
O(log n)
O(n)
O(n log n)
O(n^2)
O(n^3)
O(2^n)
Liczba operacji (log)
Rozmiar danych (n)
10^6
10^5
10^4
10^3
10^2
10^1
10^0
10^{-1}
20
40
60
80
100

Rozmiar danych (n)	O(logn)	O(n)	O(nlogn)	$O(n^2)$	$O(n^3)$	$O(2^n)$	O(n!)
10	3.3 ns	10 ns	33 ns	100 ns	1 μs	1 μs	3.6 ms
50	5.6 ns	50 ns	282 ns	2.5 μs	125 μs	13 dni	> wiek wszechświata
100	6.6 ns	100 ns	664 ns	10 μs	1 ms	> wiek wszechświata	> wiek wszechświata
1 000	10 ns	1 μs	10 μs	1 ms	1 s	INF	INF
10 000	13 ns	10 μs	133 μs	100 ms	16.7 min	INF	INF
100 000	16 ns	100 μs	1.7 ms	10 s	11.6 dni	INF	INF

Przyjęte założenie: komputer wykonuje 10^9 operacji na sekundę

SCENARIUSZE ZŁOŻONOŚCI OBLICZENIOWEJ

- Przypadek pesymistyczny (Worst-case) - maksymalny czas działania dla najgorszych danych.
- Przypadek optymistyczny (Best-case) - minimalny czas działania, rzadko analizowany.
- Przypadek średni (Average-case) - oczekiwany czas działania dla losowego rozkładu danych.

PRZYKŁADY ZŁOŻONOŚCI PAMIĘCIOWEJ

- O(1) - Algorytm wykonuje obliczenia w miejscu. Używamy tylko kilku zmiennych pomocniczych (np. zamiana dwóch liczb miejscami).
- O(n) - Musimy stworzyć kopię całej tablicy lub zapisać wyniki do nowej struktury o tej samej wielkości co dane wejściowe.
- $O(n^2)$ - Tworzenie macierzy dwuwymiarowej o wymiarach n*n (np. tabliczka mnożenia lub mapa odległości między wszystkimi miastami).

TECHNIKI SORTOWANIA

SORTOWANIE BĄBELKOWE

- Zasada działania: Polega na cyklicznym porównywaniu sąsiadujących elementów i zamienianiu ich miejscami, jeśli są w złej kolejności.
- Intuicja: W kolejnych przejściach "większe" elementy stopniowo "wypływają" na koniec tablicy (jak bąbelki).
- Złożoność obliczeniowa: Jest to jeden z najprostszych, ale najmniej efektywnych algorytmów.
- Przypadek pesymistyczny / średni: $O(n^2)$
- Przypadek optymistyczny (jeśli posortowane): $O(n)$
- Ważne: Używany głównie w celach edukacyjnych, nie zalecany dla dużych zbiorów danych.

SORTOWANIE PRZEZ WYBIERANIE

- **Zasada działania:** Polega na wielokrotnym znajdowaniu najmniejszego (lub największego) elementu z nieposortowanej części tablicy i zamianie go miejscami z pierwszym elementem tej części.
- **Intuicja:** W każdej iteracji "wybieramy" odpowiedni element i umieszczamy go na jego docelowej pozycji.
- **Złożoność obliczeniowa:** Charakteryzuje się stałą złożonością w każdym przypadku.
- **Przypadek pesymistyczny / średni / optymistyczny:** $O(n^2)$
- **Ważne:** Jest prosty w implementacji, ale bardzo nieefektywny dla dużych zbiorów danych. Ma przewagę nad Sortowaniem Bąbelkowym, ponieważ wykonuje znacznie mniej operacji zamiany.

SORTOWANIE PRZEZ WSTAWIANIE

- Zasada działania: Polega na budowaniu posortowanej podlisty poprzez branie kolejnych elementów z nieposortowanej części i wstawianie ich do posortowanej podlisty w odpowiednie miejsce.
- Intuicja: Jest podobne do sortowania kart w ręce. Wstawiasz nową kartę w już posortowany zestaw.
- Złożoność obliczeniowa: Zależy od stopnia posortowania danych.
- Przypadek pesymistyczny / średni: $O(n^2)$ (dane w odwrotnej kolejności)
- Przypadek optymistyczny (jeśli posortowane): $O(n)$ (wymaga tylko liniowego przejścia)
- Ważne: Jest efektywny dla małych zbiorów danych lub dla zbiorów częściowo posortowanych. Jest stabilny (zachowuje względną kolejność równych elementów).

DZIEL I ZWYCIĘŻAJ

- Dziel: Dzielimy duży problem na mniejsze, niezależne podproblemy.
- Zwyciężaj: Rekurencyjnie rozwiązujemy podproblemy (lub bezpośrednio, jeśli są wystarczająco małe).
- Scalaj: Łączymy rozwiązania podproblemów, aby uzyskać rozwiązanie problemu oryginalnego.

REKURENCJA

- Technika, w której funkcja lub procedura odwołuje się do samej siebie.
- Niezbędne są dwa elementy: krok rekurencyjny i warunek bazowy (stopu).
- Warunek bazowy musi być osiągnięty, aby uniknąć nieskończonej pętli.
- Często używana do rozwiązywania problemów, które można podzielić na mniejsze, podobne instancje (np. w algorytmach "Dziel i Zwyciężaj").

Sortowanie in-place

- Definicja: Algorytm sortowania, który nie wymaga dodatkowej pamięci roboczej, poza niewielką stałą ilością (O(1)).
- Charakterystyka: Modyfikuje dane bezpośrednio w oryginalnej strukturze, oszczędzając zasoby pamięci.
- Przykłady: Sortowanie Bąbelkowe, Sortowanie przez Wybieranie, Sortowanie Szybkie (w typowej implementacji).

Sortowanie out-of-place

- Definicja: Algorytm, który wymaga dodatkowej pamięci proporcjonalnej do rozmiaru danych wejściowych, najczęściej O(n).
- Charakterystyka: Używa tymczasowych struktur danych (np. tablic pomocniczych) do przechowywania i scalania posortowanych podzbiorów.
- Przykłady: Sortowanie przez Scalanie (O(n)), Sortowanie przez Wstawianie (może być in-place, ale niekiedy klasyfikowane jako out-of-place).

SORTOWANIE PRZEZ SCALANIE

- Zasada działania: Jest to algorytm typu Dziel i Zwyciężaj. Polega na rekurencyjnym dzieleniu listy na dwie połowy, dopóki nie uzyskamy pojedynczych elementów (lub list jednopunktowych), a następnie na scalaniu tych podlist w posortowany sposób.
- Intuicja: Lista jest dzielona na mniejsze, łatwe do posortowania fragmenty, które następnie są łączone (scalane) z powrotem, zachowując porządek.
- Złożoność obliczeniowa: Jest bardzo wydajny i charakteryzuje się tą samą złożonością niezależnie od stopnia posortowania danych.
- Przypadek pesymistyczny / średni / optymistyczny: O(n log n)
- Ważne: Jest algorytmem stabilnym (zachowuje względną kolejność równych elementów). Wymaga dodatkowej pamięci (złożoność pamięciowa O(n)) do przechowywania scalanych podlist.

SORTOWANIE SZYBKIE

- Zasada działania: Jest to algorytm typu Dziel i Zwyciężaj. Polega na wybraniu elementu zwanego pivotem (elementem osiowym) i podziale pozostałych elementów na dwie podtablice: te mniejsze od pivota i te większe. Następnie rekurencyjnie sortuje się podtablice.
- Intuicja: Wybranie pivota i ustawienie go na jego docelowej pozycji, tak by wszystkie elementy po jego lewej były mniejsze, a po prawej większe. To drastycznie zmniejsza rozmiar problemu.
- Złożoność obliczeniowa: Zależy od sposobu wyboru pivota i podziału.
- Przypadek pesymistyczny: $O(n^2)$ (gdy pivot jest zawsze najmniejszym/największym elementem)
- Przypadek średni / optymistyczny: $O(n \log n)$
- Ważne: Jest to najczęściej używany algorytm sortowania ze względu na bardzo dobrą wydajność w praktyce. Algorytm jest niestabilny i sortuje w miejscu, nie wymagając dodatkowej pamięci (złożoność pamięciowa $O(\log n)$ lub $O(n)$ w zależności od implementacji).

STABILNOŚĆ ALGORYTMÓW

- Definicja: Właściwość algorytmu sortowania, która oznacza, że zachowuje on względną kolejność elementów o równych kluczach (wartościach).
- Intuicja: Jeśli w zbiorze początkowym element A jest przed elementem B, a mają tę samą wartość, to po posortowaniu stabilnym algorytmem A nadal będzie przed B.
- Algorytmy Stabilne (Przykłady):
  - Sortowanie przez Wstawianie (Insertion Sort)
  - Sortowanie przez Scalanie (Merge Sort)
  - Sortowanie Bąbelkowe (Bubble Sort)
- Algorytmy Niestabilne (Przykłady):
  - Sortowanie Szybkie (Quick Sort)
  - Sortowanie przez Wybieranie (Selection Sort)
- Ważne: Stabilność jest kluczowa, gdy sortowany zbiór ma dodatkowe atrybuty, które nie są brane pod uwagę w kluczu sortowania (np. sortowanie według nazwiska, przy jednoczesnym zachowaniu pierwotnej kolejności wprowadzania dla osób o tym samym nazwisku).

DODATKOWE MATERIAŁY

- https://see-algorithms.com/sorting/QuickSort
- https://www.youtube.com/@AlgoRythmics
- https://visualgo.net/en/sorting
- https://www.bigocheatsheet.com/
- https://www.geeksforgeeks.org/dsa/sorting-algorithms/
- https://www.toptal.com/developers/sorting-algorithms
- https://www.codewars.com/
- https://leetcode.com/
- https://adventofcode.com/